

Math220 Introductory Programming For Computational Mathematics

Introduction to Python

Dr. E. K. A. Schwinger

University of Ghana
Department of Mathematics

Semester 2 (2022)



Why Python

- Ease of use and learn
- Community
- Dynamic typing
- Large number of libraries and frameworks
- Versatility
- Big data and data science
- Wide applicability



Variables

Definition

A python variable is a named reference to an object, usually in memory. A variable name is the name given to the variable. All programming languages including python have rules for naming variables.

Rules for variable names

- Variable names can only include alpha-numeric characters and underscores.
- Variable names cannot start with a number.
- Variable names must start with a letter or the underscore character.
- Variable names are case-sensitive.
- Variable names should not be python keywords.



Which of the following is not a valid python variable name

- 1student
- student-name
- std_name
- rad1us
- class+list
- class.list
- _classSize
- print



Python has a large number of data types including numeric, non-numeric and sequence.

- Numeric
 - Integers (int)
 - Floats (float)
 - Complex (complex)
 - Boolean (bool)
- Non-numeric
 - Text (str)
- Sequence
 - List (list)
 - Tuple (tuple)
 - range
- Mapping
 - Dictionary (dict)
- Set (set)
- NoneType (None)



Python Types

Python datatype examples

Integer: `myint = 20`

Float: `myfloat = 20.0`

Complex: `mycomplex = 20 + 12j`

Boolean: `mybool1 = True`, `mybool2 = False`

Text: `mystr = 'This is a Python string'`

List: `mylist = ['a', 1, 2, 3.5, 20 + 12j]`

Tuple: `mytuple = ('a', 1, 2, 3.5, 20 + 12j)`

Set: `myset = {'a', 1, 2, 3.5, 20 + 12j}`

Dictionary: `mydict = {'Course':'Math220', 'Credit':3}`

NoneType: `myint = None`



Basic Operators

Arithmetic Operators

Addition: +

Subtraction: -

Multiplication: *

Division: /

Floor division: //

Modulus: %

Exponent: **

Assignment Operators

Assignment: =

Add-assign: +=

Subtract-assign: -=

Multiply-assign: *=

Divide-assign: /=

Floor-divide-assign: //=

Modulus-assign: %=

Exponent-assign: **=



Basic Operators

Comparison Operators

Equal: `==`

Not equal: `!=`

Greater than: `>`

Greater or Equal: `>=`

Less Than: `<`

Less or Equal: `<=`

Other Operators

Logical and: `and`

Logical or: `or`

Logical not: `not`

Membership: `in`

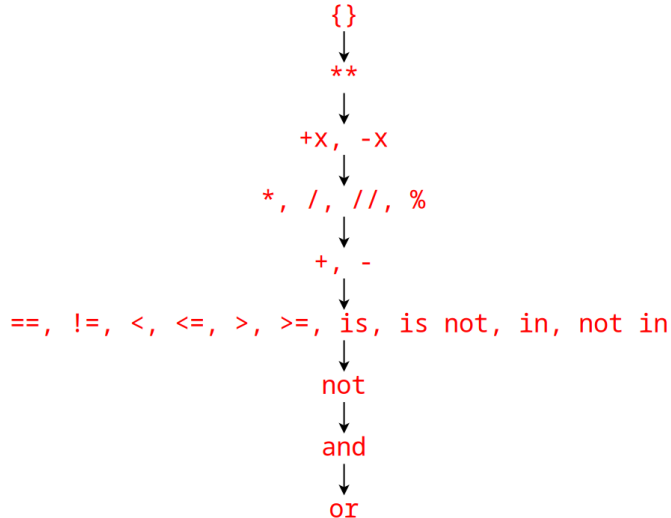
`not in`

Identity: `is`

`not is`



Operator Precedence



Complex data type

Complex Object

The complex data type is an object and just like complex numbers, it

- has real and imaginary part
- Can be conjugated
- Has magnitude

Example

```
1 myval = complex(1,3)
2 props = {'Real':myval.real, 'Imaginary':myval.imag, 'Conjugate':myval.conjugate(),
  ↪      'Magnitude':abs(myval)}
3 print(props)
```



Complex data type

Example

```
1 myval = complex(1,3)
2 props = {'Real':myval.real, 'Imaginary':myval.imag, 'Conjugate':myval.conjugate(),
  ↪      'Magnitude':abs(myval)}
3 print(props)
```

Example

```
'Real': 1.0, 'Imaginary': 3.0, 'Conjugate': (1-3j), 'Magnitude': 3.1622776601683795
```



Example

Example

Write python code that finds the roots of the quadratic equation $ax^2 + bx + c$ where $a = 1$, $b = 5$, $c = 6$



Example

Example

Write python code that finds the roots of the quadratic equation $ax^2 + bx + c$ where $a = 1$, $b = 5$, $c = 6$

Solution

```
1 a = 1
2 b = 5
3 c = 6
4 D = b**2 - 4*a*c
5 x1 = (-b + D**0.5)/(2*a)
6 x2 = (-b - D**0.5)/(2*a)
7 print({'x_1':x1,'x_2':x2})
```

Output

This gives the output 'x1': -2.0, 'x2': -3.0



Creating Functions

In the previous example we found the root of the quadratic $x^2 + 5x + 6$. With what we did, if we now want to find the roots of a new quadratic, say $3x^3 + 11x + 6$, we would have to rewrite the whole code again, changing the values of a , b , c . Functions provide us with the ability to reuse code.



Defining Functions

The syntax for creating a function is

Function definition

```
1 def function_name(arg_list):  
2     function_code 1  
3     function_code 2  
4     ....  
5     function_code n  
6     return output
```



Defining Functions

The syntax for creating a function is

Function definition

```
1 def function_name(arg_list): ← Function/Block header
2     function_code 1
3     function_code 2
4     ....
5     function_code n
6     return output
```



Defining Functions

The syntax for creating a function is

Function definition

```
1 def function_name(arg_list) :  
2     function_code 1  
3     function_code 2  
4     ....  
5     function_code n  
6     return output
```

← Function/Block header



Defining Functions

The syntax for creating a function is

Function definition

```
1 def function_name(arg_list) :  
2     function_code 1  
3     function_code 2  
4     ....  
5     function_code n  
6     return output
```

Function/Block header

Function/Block body



Defining Functions

The syntax for creating a function is

Function definition

```
1 def function_name(arg_list) :  
2     function_code 1  
3     function_code 2  
4     ....  
5     function_code n  
6     return output
```

Function/Block header

Function/Block body

Indentation



Introduction to Blocks (Creating Functions)

To convert our quadratic code to a function, we have

Quadratic definition

```
1 def quadratic(a,b,c):  
2     D = b**2 - 4*a*c  
3     x1 = (-b + D**0.5)/2  
4     x2 = (-b - D**0.5)/2  
5     return(x1,x2)
```

We can then call the function using the function call `quadratic(3,9,6)` for the quadratic equation $3x^2 + 9x + 6$



Using Functions

Apart from the functions that we create ourselves, python has a large collection of functions. Some come with the initial python installation and others have to be installed. Python functions are stored in files which contain python statements, and definitions. These files are called modules. Functions in modules have to be imported before they can become available to a program. The following are the syntax for importing python modules

- `import modulename`
- `import modulename as alias`
- `from modulename import function1, function2, function3`

For this to work, there should be a file called `modulename.py` within your python path



Using Functions

```
import modulename
```

If the module is imported using this statement, the functions within the module exist within module namespace. The function must therefore be accessed using the fully qualified name

```
modulename.functionname(parameters)
```

```
import modulename as alias
```

If the module is imported using this statement, a namespace is created with the name alias and the functions within the module exist within that namespace. The function must therefore be accessed using the fully qualified name

```
alias.functionname(parameters)
```

```
from modulename import function1, function2, function3
```

Functions imported this way exists within the global namespace and can be accessed directly

```
function1(parameters)
```



Using Functions

The `math` and `cmath` modules usually come preinstalled with the python programming language. These modules contain such mathematics functions as `sin`, `cos`, `tan`, `arctan`, `sqrt`, `exp`, and many more. They also come with the constants `pi` and `e`. The `math` module is restricted to working in the domain of real numbers while the `cmath` module works in the complex domain. Explore the module and try to know some of the functions that are in the modules. You can start by looking at the namespace that has all the default names that python is started with using `print(dir(__builtin__))`. Now import the `math` library and explore the functions in the library

```
1 import math
2 print(dir())
```



Using Functions

The previous code lists all the names that have been defined locally. Now to list all the functions in the math module, run `dir(math)`

Do the same thing with the cmath module and discuss the difference between the two modules.

Run the code `from math import sin, cos, pi`. Now list all the files in the local namespace



Using Functions Example

In algebra you learnt about the n^{th} roots of unity, defined as

$$z_k = \cos \frac{2k\pi}{n} + i \sin \frac{2k\pi}{n}, \quad k = 0, 1, \dots, n-1$$

Write code to find the values of the 5th roots of unity.



Sequence types

Sequence types

Python sequence data types are data types with collection of elements whose order depends on the order in which they were inserted. Examples are

- Strings
- Lists
- Tuples

Elements in sequence data types can be accessed through by location or more formally their index. Python is zero indexed hence the first element has index 0 and the n^{th} element has index $n - 1$.



Sequence Indexing and Slicing

Indexing

Python sequences are indexed with the square bracket `[]`. Given the sequence object `myseq` with n elements.

- The i^{th} element of `myseq` can be accessed using the python code `myseq[i-1]`.
- The `(len)` function is used to find the number of elements in a sequence. The function takes as argument the sequence whose length is being calculated.
- Python will raise `IndexError` if you try to index a list beyond the last element

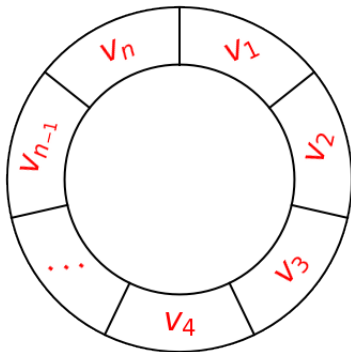


Indexing

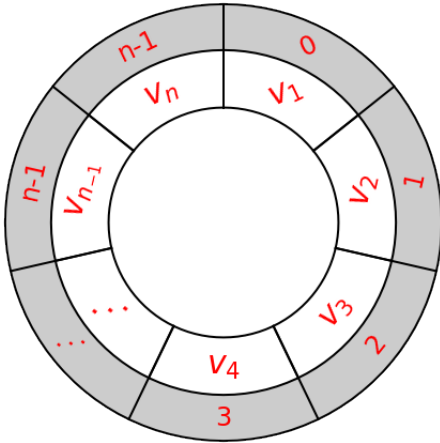
The following indexing operations will return the last, second-to-last, third-to-last and i^{th} -to-last respectively `myseq[len(myseq)-1]`, `myseq[len(myseq)-2]`, `myseq[len(myseq)-3]`, `myseq[len(myseq)-i]`. This can be written in python using the shortcut `myseq[-1]`, `myseq[-2]`, `myseq[-3]`, `myseq[-i]`.



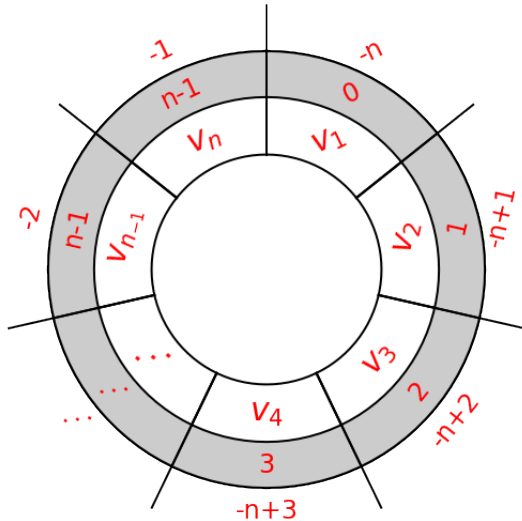
Indexing



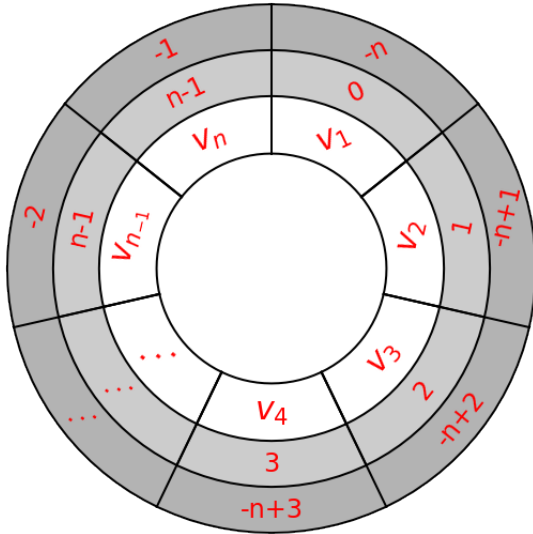
Indexing



Indexing



Indexing



Slicing

Slicing extends indexing to select a group of elements at once. Python uses the colon operator for slicing. The full slicing operator has the format `start:stop:step`. This is equivalent to selecting elements of the sequence at index values selected from the arithmetic progression beginning from `start` and stopping at the last sequence value before `stop` with common difference `step`. The format does have compact forms since `start` and `step` have default values 0 and 1 respectively. The following must be noted when using the slice operator:

- The components of the slice operator must all be integers.
- If `start < stop` then `step` must be positive.
- If `start > stop` then `step` must be negative.



Slicing

Thus the slice operator can be written in the following ways

- `myseq[start:stop:step]`
- `myseq[:stop]` where start and step are replaced by their default values.
- `myseq[start:stop]` where the default value for step is used.
- `myseq[start:]` which means select all elements beginning from start
- `myseq[::step]` which means start from the beginning to the end and select every element at interval step.
- `myseq[:::]` which is equivalent to `myseq`.

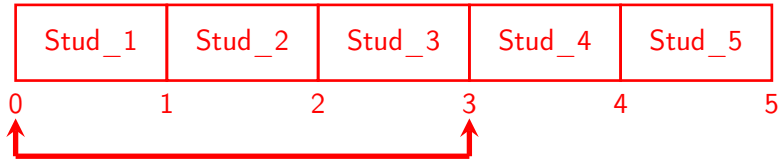


Example

Consider the list stored in the variable `stud_list` with 20 elements:

- Which elements will be returned by the code `stud_list[2:15:4]`
- Which elements will be returned by the code `stud_list[10:1]`
- Which elements will be returned by the code `stud_list[12:1:-3]`
- Which elements will be returned by the code `stud_list[::-1]`
- Which elements will be returned by the code `stud_list[2::4]`
- Which elements will be returned by the code `stud_list[:15:4]`
- How can the elements in index positions 5, 8, 11, 14 be extracted?





Schematic view of the slicing operation `std_list[0:3]`



Lists in python are used to store multiple (possibly heterogeneous) objects.

- Lists are created using the square brackets `[]`.
- Elements in a list are comma separated.
- An empty list can be created using an empty square bracket
- Lists are mutable. This means elements within a list can be updated by assignment.
- New elements can be added to lists using `append` or `extend`.
- Lists provide mechanisms for searching for elements. These include:
 - `count`
 - `index`.

The python `in` membership operator can also be used to test for the presence of elements.



- In python, assignment of a sequence variable to a new variable just creates an alias for the existing variable. This means the two variables both point to the same sequence. Lists provide a copy method to create two independent lists.



Working with lists

Type the following codes and test their outputs

Example

```
1 a = [1,2,3,4,5,6,7,8,9]
2 b = a
3 b.append(10)
4 print(a)
```

```
1 a = [1,2,3,4,5,6,7,8,9]
2 b = a.copy()
3 b.append(10)
4 print(a)
```



Working with lists

Type the following codes and test their outputs

Example

```
1 a = [1,2,3,4,5,6,7,8,9]
2 a.append([10,11,12])
3 print(a)
```

```
1 a = [1,2,3,4,5,6,7,8,9]
2 a.extend([10,11,12])
3 print(a)
```



Working with lists

Type the following codes and test their outputs

Example

```
1 a = [1,2,3,4,1,1,5,2,4]
2 #Find the first occurrence of 1 beginning from index 0
3 print({'count':a.count(1),'index':a.index(1)})
4 #Find the first occurrence of 1 beginning from index 1
5 print({'count':a.count(1),'index':a.index(1,1)})
```

```
1 a = [1,2,3,4,1,1,5,2,4]
2 #Find the first occurrence of 1 beginning from index 0
3 print({'count':a.count(1),'index':a.index(8)})
```



Question? What happens if the element you are trying to find using index does not exist in the list?



Tuples in python are used to store multiple (possibly heterogeneous) objects.

- Tuples are created using brackets `()`.
- Elements in a tuple are comma separated.
- Tuples are immutable. This means elements within a list cannot be updated by assignment.
- Immutability also means that once tuples are created, they are essentially readonly.
- Tuples also use `count` and `index` for searching for elements. The python `in` membership operator can also be used to test for the presence of elements.



Working with lists

Type the following codes and test their outputs

Example

```
1 a = [1,2,3,4,1,1,5,2,5]
2 a[5] = 2
3
4 b = (1,2,3,4,1,1,5,2,5)
5 b[5] = 2
```

```
1 a = [1,2,3,4,1,1,5,2,4]
2 print(8 in a)
```



It should be noted that in python, the addition (+) and multiplication (*) operators can also be used with lists. Adding two lists extends the first list by the second. Multiplying a list by an integer n extends the list by its original self $(n - 1)$ times. Note however that extend and append operations are performed in-place but with addition and multiplication the result must be reassigned to the result variable.



Working with Lists

Try with the following sets of code

```
1  a = [1,2,3,4,5,6]
2  b = ['a', 'b', 'c']
3  c = b.copy()
4
5  d = a + b
6  a.extend(b)
7  print(a == d)
8
9  e = 5*b
10 b.extend(e), b.extend(e), b.extend(e), b.extend(e)
11 print(b == e)
```



Working with Strings

Strings are a collection of characters. Strings are also immutable. Strings have many methods for working with them. The following are some of the string methods:

Introspection

- `isalnum`
- `isalpha`
- `isdigit`
- `isupper`
- `islower`
- `startswith`
- `endswith`
- `istitle`
- `isupper`

Search & Replace

- `index`
- `find`
- `count`
- `replace`
- `index`
- `rfind`
- `rindex`

Others

- `lstrip`
- `rstrip`
- `strip`
- `split`
- `splitlines`

Formatting

- `capitalize`
- `upper`
- `lower`
- `title`



Working with strings

String Examples

Try typing the following codes and understanding how they work. Read about and try your hands on other string methods.

```
1 a = 'This is a Python string'
2 print(a.upper())
3 print(a.lower())
4 print('P is located at index '+str(a.index('P')))
5 print(a.replace('Python','Cython'))
6 print('endswith the text String? '+ str(a.endswith('String')))
7 print('endswith the text string? '+ str(a.endswith('string')))
```



Example

Consider the mathematical structure with the coordinates $(1.0, 0.0)$, $(3.0, 5.2)$, $(-0.5, 0.87)$, $(-6.0, 0.0)$, $(-0.5, -0.87)$, $(3.0, -5.2)$. Write python code to find the circumference of the structure. How would you extend it if your structure has many points.



Looping through sequences

Python provides two statements for iterating through sequences. They are the `for` and `while` statements.

For-loop

The `for` loop is used to loop over elements in a sequence. It's syntax is as follows:

```
1  for element in sequence:
2      statement_1 (possibly on/with) element
3      statement_2 (possibly on/with) element
4      ...
5      statement_n (possibly on/with) element
```

`element` in the block header can be replaced by any valid variable name. That is a placeholder used as the identifier for any element picked from sequence.



For loop

```
1 a = [1,2,3,4,5,6,7,8]
2 for i in a:
3     print("i is "+str(i))
```

Now for a more complicated example

```
1 a = [1,2,3,4,5,6,7,8,9,10]
2 a_split = [],[]
3 for i in a:
4     a_split[i%2].append(i)
5
6 print("Even numbers are "+str(a_split[0]))
7 print("Odd numbers are "+str(a_split[1]))
```



While loop

The while loop performs a repeated set of statements so far as a specified condition remains true and exits if the condition becomes false. The while loop requires that the variable on which the condition is to be checked be created before it is checked. In most cases, python also requires that the variable be updated within the loop. The syntax of the while loop is as follows:

```
1 initialize var
2 while (condition on var):
3     statement_1
4     statement_2
5     ...
6     possibly update var
7     ...
8     statement_n
```



While loop example

```
1 a = [1,2,3,4,5,6,7,8,9]
2 i = 0
3 a_pow = []
4 while i < len(a):
5     a_pow.append(10**a[i])
6     i += 1
7 a_pow
```

It should be noted that for and while loops can on many occasions be used to perform solve the same problems.



Problem set 1

- 1 Consider a circle of area A . We know that the area of the circle is calculated as $\pi r^2 = A$. Create a function that finds the radius of a circle given the area.
- 2 A palindrome is a word, phrase, or sequence that reads the same backwards as forwards, e.g. madam or nurses run. Write python code to test if a string is a palindrome
- 3 To convert a decimal integer to a binary string, we perform successive divisions until the quotient is equal to 0 and read the remainders backwards to get the binary string. Example, to convert the number 25 to binary we have

12r 1

6r 0

3r 0

1r 1

0r 1



Problem set 1

- ④ Related to the previous example is the problem of converting a binary number to a decimal one. In this case we find the sum of the successive multiplication of the elements of the binary string by increasing powers of 2. Thus $11001_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 8 + 0 + 0 + 1 = 25_{10}$. Write a python function that converts a binary string to a decimal. Test your function on the numbers 100001001, 01010101010101 and 101100101.
- ⑤ Given a sequence $x_1, x_2, x_3, x_4, \dots, x_{n-1}, x_n$ the sequence is a geometric sequence if $\frac{x_i}{x_{i-1}} = K \quad i = 2, \dots, n$. Write a function that takes a list of numbers and determines if the list represents a geometric sequence. Test your function with the sequence (4, 15, 26, 37, 48, 59, 70) and (1, -3, 9, -27, 81, -243).
- ⑥ The vector equation of a line is given by $\underline{r} = \underline{a} + t\underline{d}$. This can be expanded as the equation $(x, y) = (a_1, a_2) + t(d_1, d_2)$ where $\underline{a}$ is a point on the line and $\underline{d}$ is the direction vector. Write a function that takes $\underline{a}$ and $\underline{d}$ as tuples and t as a float and determines the position vector of the point.



Problem set 1

- 7 Let $\underline{a} = (a_1, a_2)$ and $\underline{b} = (b_1, b_2)$ be vectors, we know that $\underline{a} \cdot \underline{b} = |\underline{a}| |\underline{b}| \cos \theta$. Find the angle between the two vectors defined by the tuple (4,3) and (5,12).
- 8 In calculus, we learn that at the turning point of a function the derivative is 0. In the face of minor floating point errors, we sometimes are not able to find explicit zeros in the numerical approximation of the derivative. But assuming the derivative changes sign, then with the assumption that the derivative is continuous, the intermediate value theorem guarantees a zero between the two points. We can therefore reduce the problem of finding a zero in a sequence to finding a change of sign. Given the sequence $[1, 1, 1, 1, -1, -1, 1, -1, -1, 1]$, write python code to determine if there is a change of sign within the sequence.



Decision making

There are times in programming when decisions have to be made on what step to be taken next. That is done in python using the `if/elif/else` blocks.

- `if` The `if` block is the first/basic part of the decision making block. The `if` block has the first condition to be tested. This block can exist alone unlike the other blocks.
- `elif` The `elif` blocks also have conditions attached. An `elif` block is tested if the previous conditions are false. There can be many `elif` blocks after the `if` block and each of them will come with its own condition. An `elif` block cannot exist without an `if` block
- `else` The `else` block catches anything that was not caught by the previous conditions. There can be only one `else` block and that should be the last block in the group.



if syntax

```
1  if condition:
2      true_statement_1
3      true_statement_2
4      ...
5      true_statement_n
```

In this case, the statements in the if block will be executed only if condition is true. If condition is false, the block is ignored.



if-else

```
1  if condition:
2      true_statement_1
3      true_statement_2
4      ...
5      true_statement_n
6  else:
7      false_statement_1
8      false_statement_2
9      ...
10     ...
11     false_statement_m
```

In this case, the statements in the if block will be executed only if condition is true. If condition is false, the statements in the else block is executed.



if-elif-else

```
1  if condition_1:
2      true_statement_11
3      ...
4      true_statement_1n
5  elif condition_2:
6      true_statement_21
7      ...
8      true_statement_2p
9  elif condition_3:
10     true_statement_31
11     ...
12     true_statement_3t
13 else:
14     false_statement_1
15     ...
```

In this case, python will test condition_1, if it is true, the statements in the if block are executed and then python will exit the whole if-elif-else combination. If condition_1 is false then python will test condition_2 in the first elif block if it true, the statements in the block are executed, if its false, python moves to test condition_3. If all the conditions are false, then python will execute the statements in the else block.



break and continue are two statements that can be used to interrupt loops.

break The break statement is used to exit a loop even when the condition is still true.

continue The continue statement cancels execution of the remainder of the current iteration and begins with the next iteration.



break and continue

break

```
1 a = [1,2,3,4,5,6,7,8,9]
2 i = 0
3 a_pow = []
4 while i < len(a):
5     val = 10**(-a[i])
6     if val < 0.001:
7         break
8     a_pow.append(10**(-a[i]))
9     i += 1
10 a_pow
```



break and continue

continue

```
1 a = [1,2,3,4,5,6,7,8,9]
2 i = 0
3 a_pow = []
4 while i < len(a):
5     val = 10**(-a[i])
6     if val < 0.001:
7         continue
8     a_pow.append(10**(-a[i]))
9     i += 1
10 a_pow
```



Range function and object

- The `range(start, stop, step)` function generates a sequence of integers from start to stop in increments(decrements) of step.
- The default for start is 0, the default for step is 1.
- This means we can write

```
1  range(10)    #range(stop)
2  range(1,12)  #range(start, stop)
3  range(1,15,4) #range(start, stop, step)
```

- The range function can be used as a counter and index for for an list
- The parameters of the range function can only integers.



List Comprehension

- List comprehensions provide a shorter syntax for creating new lists based on existing lists.
- List comprehensions allow for the incorporation of for loops and if blocks without the need for creating blocks.
- List comprehensions are enclosed in square brackets.
- The syntax for basic list comprehensions

```
1 [expression for item in list]
```

- List comprehensions can also include conditions. The most common way to add conditions to a list comprehension is to add it to the end of the expression.

```
1 [expression for item in list if condition]
```



List comprehension

List comprehension can also include conditions where we can take an action if condition is false. In that case we have the syntax

1

```
[expression if condition else expression2 for item in list]
```

Consider the problem of approximating the function $f(x) = 3x^2 - 5x + 6$ with $x \in [-1, 3]$. To approximate this we can create the discrete interval $-1, -0.75, -0.5, \dots, 3$. Then for each element in x we apply the function. How can we generate such functions in python? How can we extend this to account for the piecewise function

$$f(x) = \begin{cases} \frac{\sin x}{x} & x \neq 0 \\ 1 & \text{otherwise} \end{cases}$$



Lambda statement

- We know now that to create functions we use the def statement to define the function and assign it a name and also define what parameters it accepts.
- Python has another mechanism for creating functions called the lambda statement.
- Functions created using the lambda statement are called anonymous functions because the lambda statement provides no mechanism for naming the function.
- Lambda functions can take multiple parameters just like normal def functions but lambda functions are one-line expressions.
- The lambda function evaluates the single statement and returns the result.
- The syntax of a lambda statement is

```
1  lambda arguments: expression
```



Anonymous functions

- Create a function that calculates $e^{-x^2} \sin(x)$
- Write a function that takes two lists/sets and determines if one is a subset of the other.
- Write a function for finding the determinant of a 2×2 matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$



- So far we have learnt to take a list/sequence and iterate over its elements to perform some operation to arrive at a result
- map, filter, reduce are three python functions that allow us to write very elegant and concise codes especially involving sequences.
- They allow us to work on lists without having to iterate through the elements. We will take each of them and describe them in the next few slides.
- All these functions take two inputs; the first being a function and the second a list that the function will work on.



The map function allows us process and transform all elements in a sequence without explicitly using for loops. It applies/maps the given function to every element of the given sequence. The syntax of the map function is

```
1 map(function, sequence)
```

The output of the function is a map object. This can be cast to a list to see the elements of the result. The function passed to the map function should take a single input.



The filter function allows us to process a sequence and extract elements that satisfy a given condition. This is the filtering process. The syntax of the filter function is

```
1 filter(function, sequence)
```

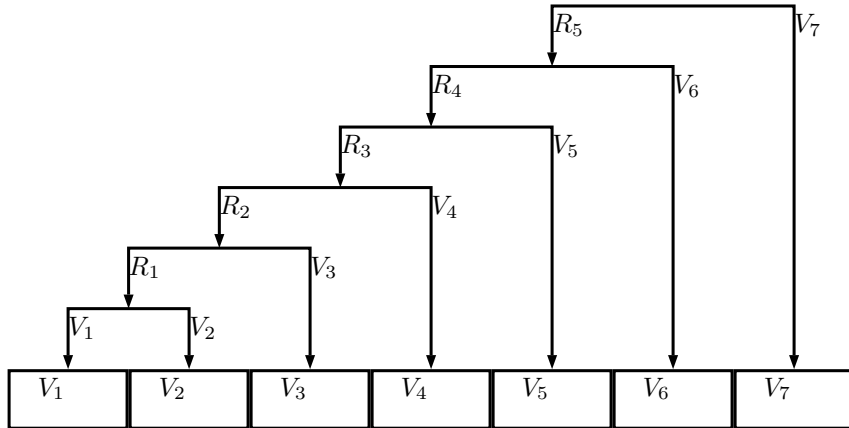
The output of filter is a filter object. The function passed as input to the filter function should return a boolean output. The element returned by filter will be those which evaluated to True.



- The reduce function can be found in the functools module.
- This function is an accumulation function.
- It takes as input a function and a sequence just like the map and filter functions.
- The function passed to reduce takes two arguments. The reduce function works by applying the input function to the elements in a pairwise manner and returns a single output which is an accumulation of all the elements in the sequence under the rule defined by the given function.
- The syntax of the reduce function is

```
1 reduce(function, sequence)
```





- The python `any()` and `all()` functions
- Python iterators and the `iter()`, `next()` functions
- Exception handling
- File IO. Reading and writing files

